



Assignment 3

Bridge Pattern

SE-2524

Ulzhan Musakhan

Astana 2026

Submission identification

- GitHub repository URL: <https://github.com/httpskwx/bridge-pattern>
- Code extension commit: 1af58ffe8bb4e64e90f84ef412add9054a67e7dc
- Topic A - Drawing
- Base commit 4c106fd7c07b6c2eb8fac8c1788288c678e3b15b
- Environment Java JDK 17

1. Design overview

Two independent dimensions. The abstraction dimension is the shape hierarchy (Shape, Circle, Square). The implementation dimension is the rendering hierarchy (Renderer, VectorRenderer, RasterRenderer, AsciiRenderer). Bridge is appropriate because these dimensions can vary independently and are linked through an interface-typed reference instead of combination subclasses. Bridge roles. Shape is the Abstraction; Circle and Square are Refined Abstractions; Renderer is the Implementor; VectorRenderer, RasterRenderer, and AsciiRenderer are Concrete Implementations; Main is the Client. The bridge itself is the field protected `Renderer` stored by Shape and supplied through its constructor.

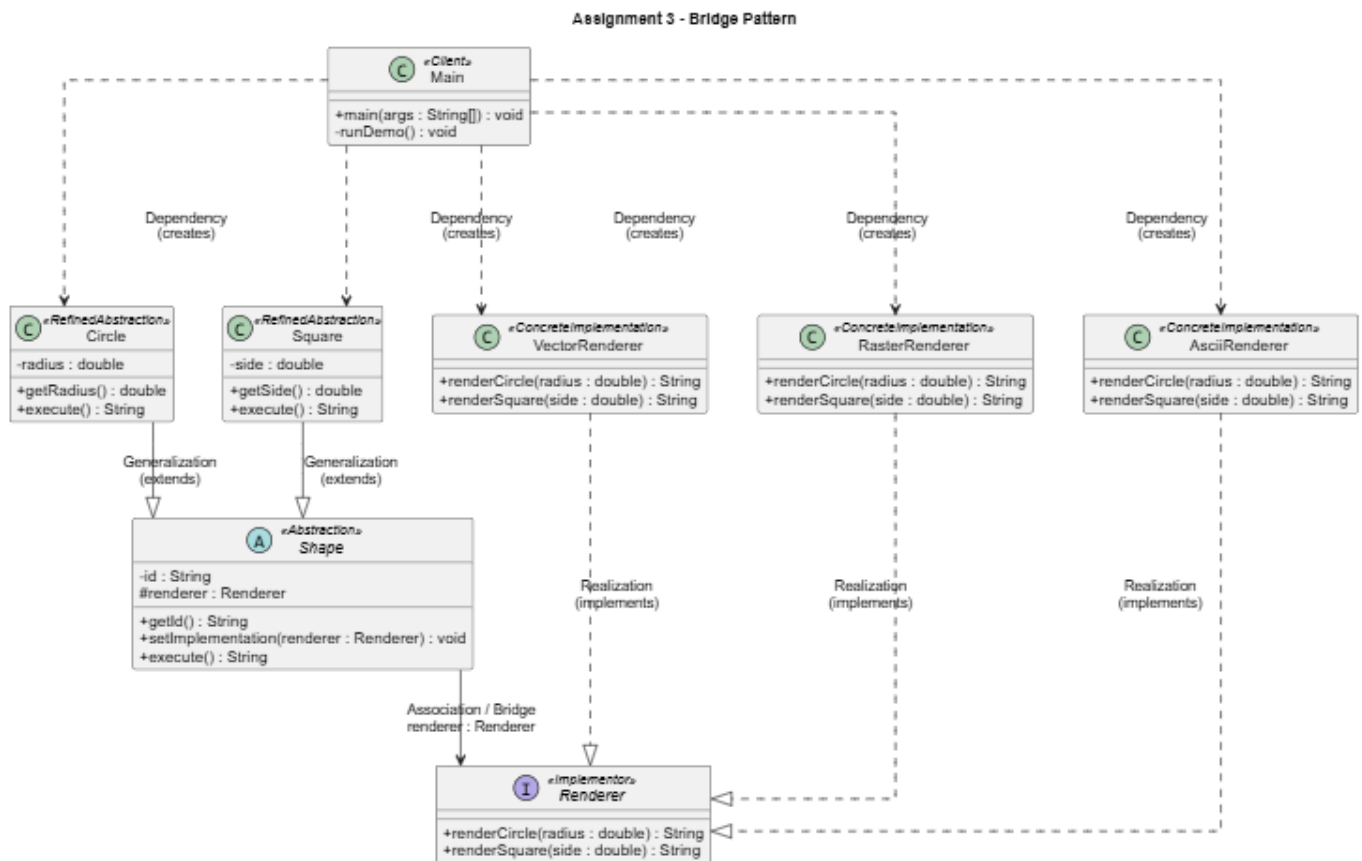


Figure 1. Final Bridge structure matching the submitted Java code.

2. Clean Code evidence in the implementation

The following excerpts identify five required Clean Code practices and their concrete benefit in this project.

2.1 Separation of responsibilities

```
1 public interface Renderer { 27 usages  httpskwikxsi
2     String renderCircle(double radius); 1 usage 3 implementations  httpskwikxsi
3     String renderSquare(double side); 1 usage 3 implementations  httpskwikxsi
4 }
```

Benefit: The interface defines low-level rendering operations, while Shape subclasses keep shape-specific data and control flow. Rendering concerns are therefore separated from abstraction concerns.

2.2 Meaningful names

```
16 public void setImplementation(Renderer renderer) { 1 usage  httpskwikxsi
17     this.renderer = renderer;
18 }
19
```

Benefit: The method and parameter names state exactly what changes: the implementation reference. This makes the runtime switch behavior readable without extra comments.

2.3 Small, focused methods

```
14 @Override 8 usages  httpskwikxsi
15 public String execute() {
16     return renderer.renderCircle(radius);
17 }
18 }
```

Benefit: Circle.execute() performs one task: delegate circle rendering with the stored radius. Its short body stays at one abstraction level.

2.4 Avoidance of duplication

```
146 @ private static boolean printResult( 6 usages  httpskwikxsi
147     String testId,
148     String classes,
149     String actual,
150     String expected
151 ) {
152
153     boolean passed = actual.equals(expected);
```

Benefit: T1-T4, T6, and T7 reuse one comparison/printing workflow instead of duplicating PASS/FAIL formatting and expectedvalue logic for every check.

2.5 Encapsulation of state

```
3 private final String id; 2 usages
```

```

3      private final double radius;  3 usages

12     public String getId() {  2 usages  https://www.kwiksi.com
13         return id;
14     }

10     public double getRadius() {  2 usages
11         return radius;
12     }

```

Benefit: Identity and domain data are private and immutable after construction. T5 can verify them through getters while preventing external code from changing the state directly.

Result. These practices keep the two Bridge hierarchies understandable, testable, and independently extensible while avoiding concrete-implementation dependencies in the abstraction side.

3. Runtime switching, extension, and design evaluation

3.1 Runtime replacement on the same object

```

Circle circle = new Circle("C3", 2, vector);
Circle originalReference = circle;
String before = circle.execute();
circle.setImplementation(raster);
String after = circle.execute();
boolean sameObject = originalReference == circle;

```

T5 proves that only the implementation reference changes. Reference equality using `==` is true, the Circle ID and radius stay unchanged, and `execute()` changes from VECTOR output to RASTER output. This demonstrates runtime flexibility without recreating the abstraction object.

3.2 Independent implementation extension

The base solution was committed before I3 at 4c106fd7c07b6c2eb8fac8c1788288c678e3b15b. Then `AsciiRenderer` was added and `Main` was updated for T6-T7. The existing `Shape`, `Circle`, `Square`, `Renderer`, `VectorRenderer`, and `RasterRenderer` sources stayed unchanged. This is visible in `extension.diff` and demonstrates the Open/Closed intent of Bridge: a new implementation can be introduced without modifying the existing abstraction hierarchy or contract.

```

public class AsciiRenderer implements Renderer {
    public String renderCircle(double radius) {
        return "ASCII circle radius=" + radius;
    }
    public String renderSquare(double side) {
        return "ASCII square side=" + side;
    }
}

```

3.3 Trade-off and Bridge vs Adapter

Trade-off. Bridge adds an extra interface, more classes, and one level of indirection. For a very small system with only one fixed renderer, this structure could be unnecessary overhead; it becomes valuable when both dimensions are expected to vary.

Bridge vs Adapter. Bridge is designed from the start to separate an abstraction from its implementation so both hierarchies can evolve independently. Adapter is used to make already existing incompatible interfaces work together. In this project there is no incompatible legacy interface to translate; `Shape` and `Renderer` are intentionally designed as two coordinated hierarchies, so Bridge is the correct intent.

3.4 Demonstration result

```
T1-T4: all Shape/Renderer base combinations PASS
T5: sameObject=true, stateUnchanged=true
T6: Circle + AsciiRenderer PASS
T7: Square + AsciiRenderer PASS
SUMMARY: 7/7 PASS
```

The project also compiles and runs using the required JDK 17 commands without IDE-specific dependencies:

```
javac --release 17 -encoding UTF-8 -d out "@sources.txt"
java -cp out Main --demo
```

References

Astana IT University, Assignment 3 | Bridge Pattern, Software Design Patterns, 2026-2027.

Course Lecture 4, The Structural Design Pattern - Bridge Pattern.

Course Lecture 3, Structural Design Pattern - The Adapter.

Robert C. Martin, Clean Code: A Handbook of Agile Software Craftsmanship, Prentice Hall, 2008